

UCUENCA

FACULTAD DE INGENIERÍA

MÓDULO DE REDES COMPLEJAS — UNIVERSIDAD DE CUENCA

ACTIVIDAD PRÁCTICA EN PAREJAS

FLUJO MÁXIMO

Ford-Fulkerson vs. Edmonds-Karp

Capítulo de Optimización en Redes — Implementaciones interactivas en Julia

DOCUMENTO ACADÉMICO — USO EN CLASE

Fabián Astudillo-Salinas — La U que vive la educación

Cuenca — Ecuador 14 de julio de 2026

Índice

1	Información General de la Actividad	2
1.1	Ficha de la actividad	2
1.2	Objetivos de aprendizaje	2
2	Fundamento Teórico	4
2.1	El problema de flujo máximo	4
2.2	Ford-Fulkerson y Edmonds-Karp	4
3	Preparación del Entorno	5
3.1	Obtención del código	5
3.2	Estructura del código base	5
4	Desarrollo de la Actividad	7
4.1	Parte 1 — Exploración guiada (2 puntos)	7
4.2	Parte 2 — El experimento zigzag (3 puntos)	7
4.3	Parte 3 — Su propia red (3 puntos)	8
4.4	Parte 4 — Análisis comparativo (2 puntos)	9
5	Entregables, Rúbrica y Condiciones	11
5.1	Entregables	11
5.2	Rúbrica de evaluación	11

Capítulo 1

Información General de la Actividad

1.1 Ficha de la actividad

Cuadro 1: Ficha general de la actividad

Campo	Detalle
Módulo	Redes Complejas
Capítulo	Optimización en redes
Tema	Flujo máximo: Ford-Fulkerson y Edmonds-Karp
Modalidad	Trabajo en parejas
Duración estimada	1 sesión de laboratorio (2 horas) + trabajo autónomo
Lenguaje	Julia (≥ 1.9) con Plots.jl y Pluto
Repositorio	https://github.com/fabianastudillo/ComplexNetworks
Código base	optimization/ford-fulkerson/ y optimization/edmonds-karp/
Entregable	Informe PDF + código Julia + animaciones GIF
Entrega	Una semana después de la sesión de laboratorio

1.2 Objetivos de aprendizaje

Objetivo general

Comprender, mediante experimentación con implementaciones interactivas en Julia, las **ventajas, desventajas, similitudes y diferencias** entre el algoritmo de Ford-Fulkerson (búsqueda de caminos aumentantes con DFS) y su refinamiento Edmonds-Karp (búsqueda con BFS), en el contexto del problema de flujo máximo en redes complejas.

Al finalizar la actividad, cada pareja será capaz de:

1. Explicar los conceptos de red de flujo, red residual, camino aumentante, cuello de botella y corte mínimo.
2. Ejecutar y modificar las implementaciones en Julia de ambos algoritmos, interpretando sus animaciones paso a paso.

3. Comparar experimentalmente el número de iteraciones y las longitudes de los caminos aumentantes de cada método.
4. Argumentar cuándo la elección del camino aumentante es determinante para la eficiencia y la terminación del algoritmo.
5. Relacionar el teorema *max-flow min-cut* con problemas de redes complejas: robustez, conectividad y detección de cuellos de botella.

 Integridad académica

El trabajo se desarrolla en parejas y cada pareja debe entregar un informe propio. Se permite discutir ideas entre parejas, pero **no** compartir texto, código modificado ni resultados. Todo material externo utilizado debe citarse correctamente.

Capítulo 2

Fundamento Teórico

2.1 El problema de flujo máximo

Una **red de flujo** es un grafo dirigido $G = (V, E)$ con una función de capacidad $c(u, v) \geq 0$, un nodo fuente s y un nodo sumidero t . Un **flujo** es una función $f(u, v)$ que respeta las capacidades ($f(u, v) \leq c(u, v)$) y conserva el flujo en los nodos intermedios (todo lo que entra sale). El problema de **flujo máximo** consiste en maximizar el valor $|f|$ que sale de s y llega a t .

La herramienta central de ambos algoritmos es la **red residual**: dado un flujo f , la capacidad residual es

$$r(u, v) = c(u, v) - f(u, v),$$

donde los *arcos de retroceso* (con $c(u, v) = 0$ pero $f(v, u) > 0$) permiten *cancelar* flujo enviado previamente. Un **camino aumentante** es un camino de s a t en la red residual; su **cuello de botella** Δ es la mínima capacidad residual a lo largo del camino.

2.2 Ford-Fulkerson y Edmonds-Karp

El **método de Ford-Fulkerson** (1956) repite un ciclo simple: mientras exista un camino aumentante en la red residual, aumentar el flujo en Δ a lo largo de él. El método no especifica *cómo* encontrar el camino; la implementación del repositorio usa DFS para la variante clásica (metodo=:dfs).

El **algoritmo de Edmonds-Karp** (1972) es la especialización que busca el camino aumentante **siempre con BFS**, es decir, el camino con *menos arcos*. Este cambio aparentemente menor tiene consecuencias profundas que ustedes verificarán experimentalmente en esta actividad.

Pregunta guía de la actividad

Si ambos algoritmos ejecutan el mismo ciclo de aumento y llegan al mismo flujo máximo, ¿por qué la forma de *elegir* el camino aumentante cambia la garantía de terminación y la complejidad de $O(E \cdot |f^*|)$ a $O(V \cdot E^2)$? Toda la actividad está diseñada para que puedan responder esta pregunta con evidencia experimental propia.

Al terminar cualquiera de los dos algoritmos, el conjunto S de nodos alcanzables desde s en la red residual define el **corte mínimo** $(S, V \setminus S)$, cuya capacidad es exactamente el flujo máximo (teorema *max-flow min-cut*).

Capítulo 3

Preparación del Entorno

3.1 Obtención del código

```
1 git clone https://github.com/fabianastudillo/ComplexNetworks.git
2 cd ComplexNetworks/optimization/ford-fulkerson
3 julia --project=. -e 'using Pkg; Pkg.instantiate()'
4 cd ../edmonds-karp
5 julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Listing 3.1: Clonación del repositorio e instalación de dependencias

3.2 Estructura del código base

Cuadro 2: Archivos principales del código base

Archivo	Contenido
ford-fulkerson/ford_fulkerson.jl	Algoritmo con búsqueda BFS o DFS (metodo=:bfs / :dfs), corte mínimo, dibujo de la red de flujo y de la red residual, animación GIF y modo interactivo.
ford-fulkerson/ejemplo1.jl	Red clásica de Cormen et al. (CLRS, flujo máximo 23): traza en consola y generación de GIF.
edmonds-karp/edmonds_karp.jl	Edmonds-Karp con registro de niveles BFS, animación de la <i>onda BFS</i> capa por capa y tabla de longitudes de caminos aumentantes.
edmonds-karp/ejemplo1.jl	Red CLRS + red <i>zigzag</i> con arco trampa de capacidad 1.
*/notebook_pluto.jl	Notebooks Pluto interactivos con <i>sliders</i> para recorrer las iteraciones.

✓ Tres formas de explorar los algoritmos

1. **Consola:** `julia --project=. ejemplo1.jl` imprime la traza y genera los GIF.
2. **Modo interactivo:** desde el REPL, `ford_fulkerson_interactivo(red, s, t)` o `edmonds_karp_interactivo(red, s, t)` avanzan fotograma a fotograma.

ma con [Enter] — ideal para discutir en pareja qué ocurrirá en el siguiente paso *antes* de verlo.

3. **Notebook Pluto:** `Pluto.run()` y abrir `notebook_pluto.jl` para experimentar con *sliders*.

Capítulo 4

Desarrollo de la Actividad

4.1 Parte 1 — Exploración guiada (2 puntos)

Ejecuten `ejemplo1.jl` de **ambos** proyectos y recorran las ejecuciones con el modo interactivo. Respondan en el informe:

1. Para la red CLRS, tabulen por iteración: camino aumentante, longitud (número de arcos), cuello de botella Δ y flujo acumulado, para BFS y para DFS por separado.
2. Identifiquen en las animaciones una iteración donde el camino aumentante use un **arco de retroceso** (punteado naranja). Expliquen con sus palabras qué flujo se está cancelando y por qué el resultado sigue siendo válido.
3. En la animación de Edmonds-Karp, observen la **onda BFS**: ¿qué representa el número d anotado bajo cada nodo? ¿Por qué el camino resaltado siempre tiene exactamente $d(t)$ arcos?
4. Comparen el último fotograma de ambos algoritmos: ¿el corte mínimo es el mismo? ¿El flujo máximo es el mismo? ¿Los flujos arco por arco son los mismos? Expliquen cada respuesta.

4.2 Parte 2 — El experimento zigzag (3 puntos)

La red *zigzag* tiene un arco trampa $u \rightarrow v$ de capacidad 1 entre dos rutas de capacidad M . En teoría, un Ford-Fulkerson que siempre eligiera el camino que atraviesa ese arco necesitaría $2M$ iteraciones.

```
1 include("ford_fulkerson.jl")
2
3 M = 1000
4 Cz = zeros{Int, 4, 4}
5 Cz[1,2] = M; Cz[1,3] = M # →su, →sv
6 Cz[2,3] = 1 # →uv (arco trampa)
7 Cz[2,4] = M; Cz[3,4] = M # →ut, →vt
8 red_z = RedFlujo(Cz, ["s", "u", "v", "t"],
9                 [(0.0, 1.0), (1.2, 2.0), (1.2, 0.0), (2.4, 1.0)])
10
11 # Comparar ambos métodos:
12 ford_fulkerson(red_z, 1, 4; metodo=:dfs)
13 ford_fulkerson(red_z, 1, 4; metodo=:bfs)
```

Listing 4.1: Red zigzag en el proyecto ford-fulkerson (ejecutar desde esa carpeta)

1. Ejecuten ambos métodos para $M \in \{10, 100, 1000, 10000\}$ y tabulen el número de iteraciones y los caminos usados en cada caso.
2. ¿La implementación DFS del repositorio alcanza el peor caso teórico de $2M$ iteraciones? Analicen el orden de exploración de la función `buscar_camino_dfs` y expliquen *por qué* se observa el número de iteraciones obtenido.
3. Propongan y justifiquen una modificación del orden de exploración de la DFS (por ejemplo, invertir el orden de los vecinos) que *empeore* su comportamiento en esta red. Impleméntenla y reporten el efecto.
4. ¿Por qué Edmonds-Karp es inmune a este problema sin importar el valor de M ? Relacionen su respuesta con la longitud de los caminos.

! El caso patológico que motiva a Edmonds-Karp

Con capacidades enteras, Ford-Fulkerson siempre termina, pero su número de iteraciones puede depender del *valor* de las capacidades, no solo del tamaño del grafo. Con capacidades irracionales existe una red (Zwick, 1995) en la que Ford-Fulkerson con mala elección de caminos **no termina nunca** y ni siquiera converge al flujo máximo. Edmonds-Karp elimina ambos problemas: su cota $O(V \cdot E^2)$ es independiente de las capacidades.

4.3 Parte 3 — Su propia red (3 puntos)

Cada pareja debe diseñar una red de flujo **original** que cumpla:

- Al menos **8 nodos** y **12 arcos**, con posiciones legibles.
- Al menos un par de arcos antiparalelos ($u \rightarrow v$ y $v \rightarrow u$).
- Que **al menos una** ejecución (BFS o DFS) use un arco de retroceso en algún camino aumentante.
- Que el número de iteraciones de BFS y DFS sea **diferente**.

Con su red: ejecuten ambos algoritmos, generen las animaciones GIF con las funciones `animar_ford_fulkerson` y `animar_edmonds_karp`, verifiquen *a mano* el corte mínimo reportado (sumen las capacidades de las aristas del corte) e inclúyanlo todo en el informe.

i Sugerencia

Diseñen primero la red en papel pensando dónde *quieren* que aparezca el cuello de botella, y luego ajusten capacidades hasta provocar los comportamientos pedidos. Documenten en el informe los intentos fallidos: el proceso de diseño también se evalúa.

4.4 Parte 4 — Análisis comparativo (2 puntos)

Con la evidencia de las partes 1–3, completen la tabla 3 y respondan las preguntas de discusión.

Cuadro 3: Tabla comparativa a completar por la pareja

Criterio	Ford-Fulkerson (DFS)	Edmonds-Karp (BFS)
Estrategia de búsqueda del camino aumentante		
Complejidad teórica		
¿Termina con capacidades irracionales?		
Iteraciones observadas (red CLRS)		
Iteraciones observadas (zigzag, $M = 10^4$)		
Longitudes de los caminos aumentantes		
Sensibilidad al valor de las capacidades		
Flujo máximo obtenido		
Corte mínimo obtenido		

Preguntas de discusión (respondan con argumentos y evidencia):

1. **Similitudes:** ¿qué comparten exactamente ambos algoritmos (invariantes, estructura del ciclo, resultado final)? ¿Qué teorema garantiza que ambos llegan al mismo valor de flujo?
2. **Diferencias:** en Edmonds-Karp las longitudes de los caminos aumentantes nunca decrecen (verifíquelo en sus tablas). ¿Se cumple esa propiedad en sus ejecuciones con DFS? ¿Qué papel juega esta propiedad en la demostración de la cota $O(V \cdot E^2)$?
3. **Ventajas y desventajas:** ¿existe algún escenario donde la variante DFS pueda ser preferible en la práctica (por ejemplo, memoria, simplicidad, redes con estructura conocida)? Justifiquen.
4. **Redes complejas:** propongan dos aplicaciones del flujo máximo / corte mínimo en el análisis de redes complejas (por ejemplo: robustez de una red de comunicaciones ante fallas, capacidad de una red eléctrica o de transporte, detección de

comunidades vía cortes). Para una de ellas, indiquen qué representarían s , t y las capacidades.

Capítulo 5

Entregables, Rúbrica y Condiciones

5.1 Entregables

1. **Informe PDF** (máximo 12 páginas sin contar anexos) con las tablas, respuestas, capturas de las animaciones y análisis de las partes 1–4. Debe incluir nombres y correos institucionales de ambos integrantes.
2. **Código Julia**: los archivos creados o modificados (red propia, DFS modificada de la parte 2), ejecutables con `julia --project=.`
3. **Animaciones**: los GIF de la red propia generados con ambos algoritmos.

5.2 Rúbrica de evaluación

Cuadro 4: Rúbrica de evaluación (sobre 10 puntos)

Componente	Criterio de calidad	Puntos
Parte 1 — Exploración guiada	Tablas completas y correctas; explicación clara de los arcos de retroceso y de la onda BFS.	2
Parte 2 — Experimento zigzag	Mediciones para los cuatro valores de M ; análisis del orden de exploración; modificación de la DFS implementada y explicada.	3
Parte 3 — Red propia	Red que cumple los cuatro requisitos; corte mínimo verificado a mano; animaciones incluidas.	3
Parte 4 — Análisis comparativo	Tabla comparativa completa; respuestas argumentadas con evidencia experimental propia.	2
Total		10

Penalizaciones

Informes que no compilen las afirmaciones con evidencia propia (tablas, capturas, código), o código que no ejecute con `julia --project=.`, perderán hasta el 50 % del puntaje del componente correspondiente. La entrega tardía se penaliza según

el reglamento del módulo.

 Criterio de excelencia

Las mejores parejas no se limitan a reportar números: explican *por qué* ocurren. Un informe sobresaliente conecta cada observación experimental con el argumento teórico correspondiente (lema de longitudes no decrecientes, teorema max-flow min-cut, dependencia de $|f^*|$ en la cota de Ford-Fulkerson).